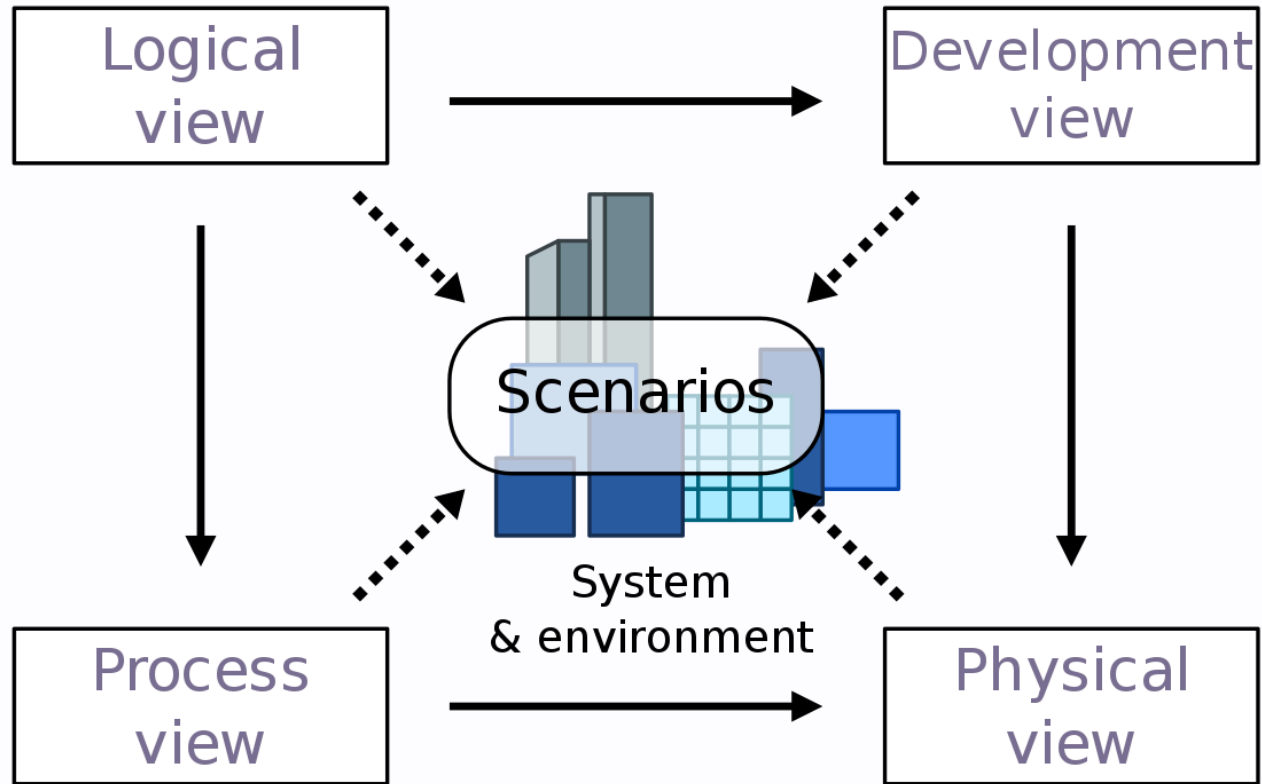


Unidad 5

Despliegue

**A diferencia de un monolítico,
desplegar una aplicación en
microservicio no es algo directo.**



Modelo de arquitectura 4+1 de Kruchten.



De la lógica a la física

Una vista lógica de una arquitectura generalmente abstrae las preocupaciones de implementación física subyacente. En este capítulo nos vamos a preocupar también de la vista física de la arquitectura.

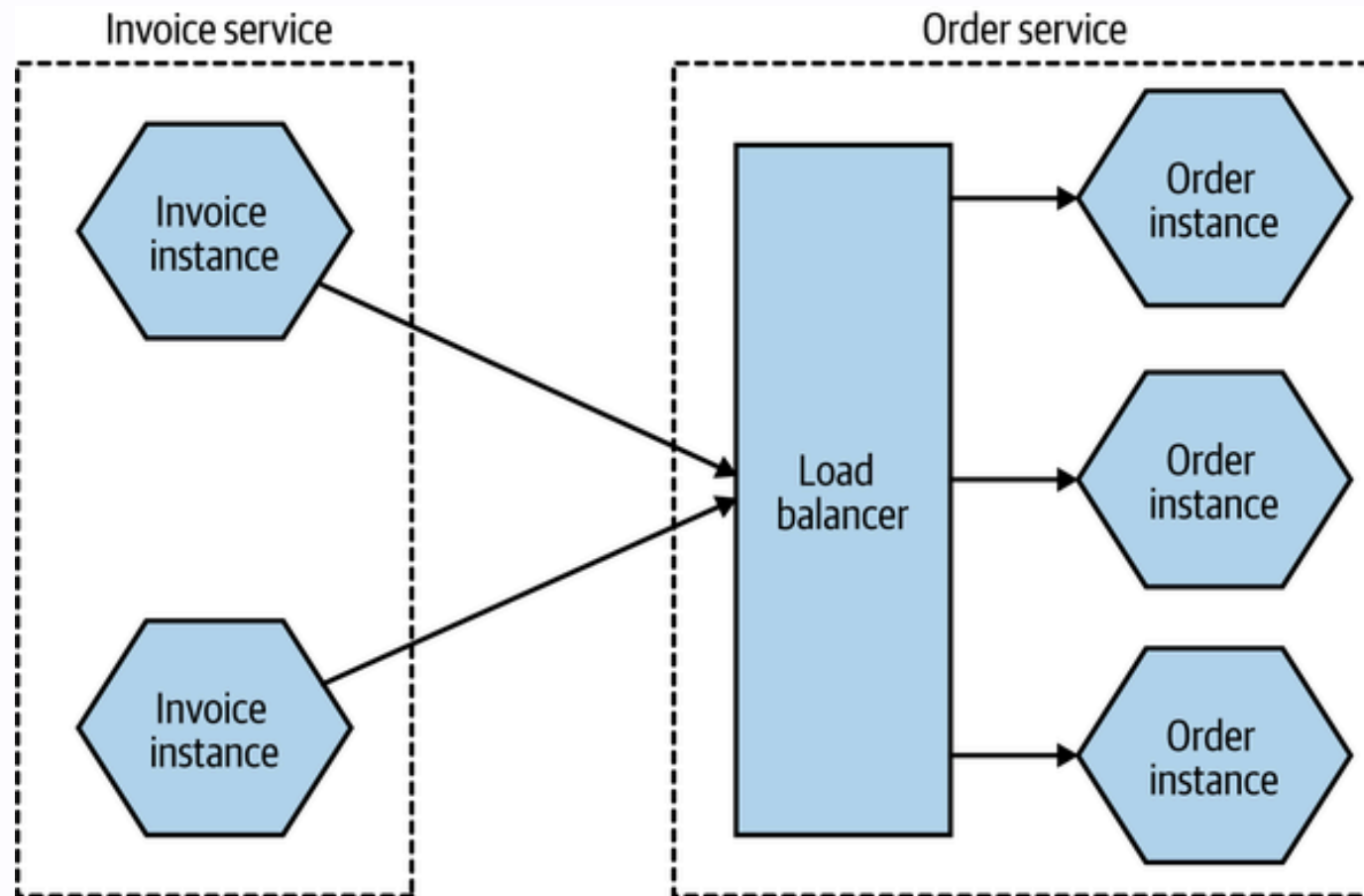


Múltiples instancias

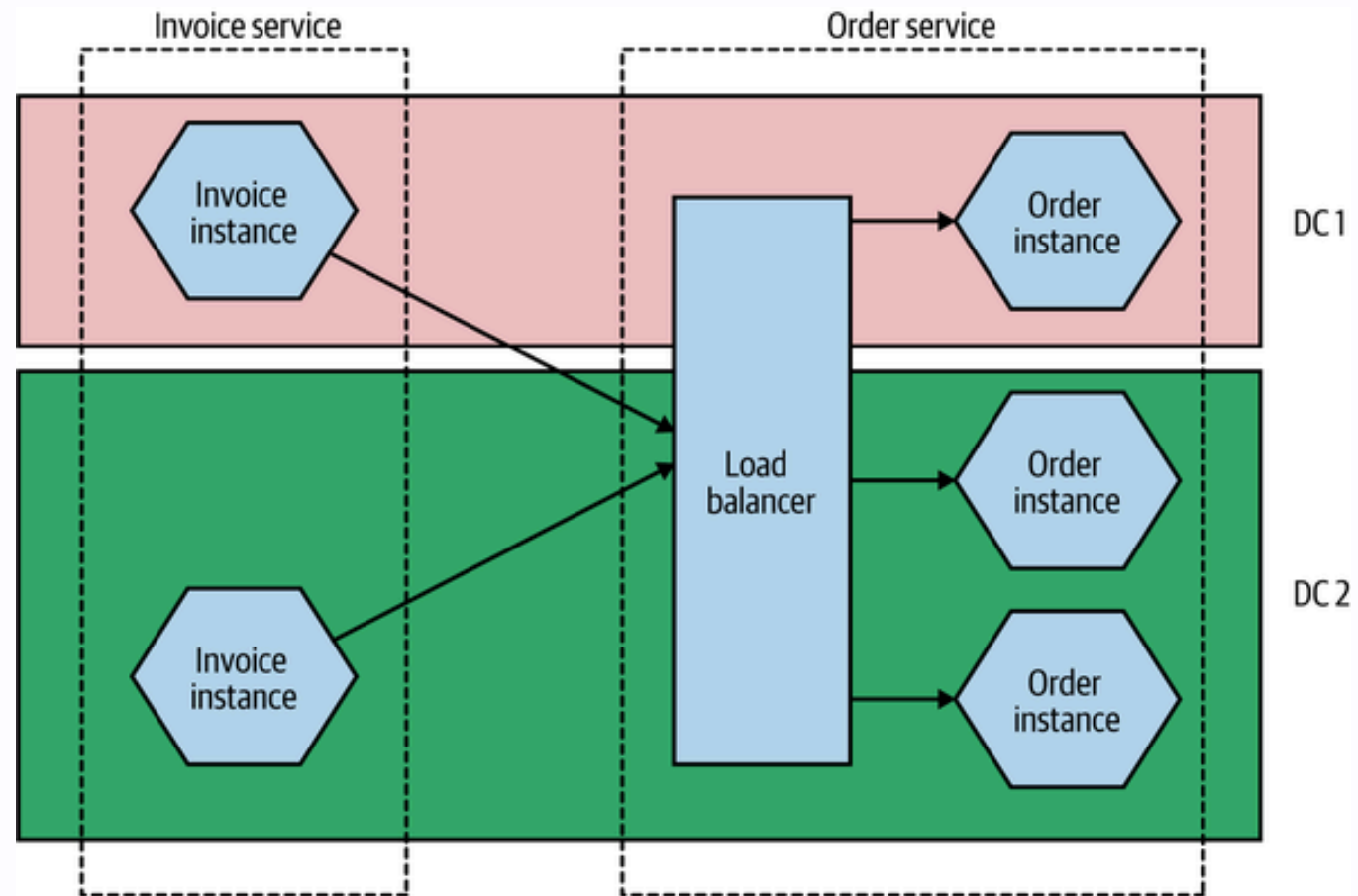
La topología de implementación (vista física) de dos microservicios no es directa.

Tener múltiples instancias permite balancear la carga y mejorar la robustez (si falla una instancia, hay otra que puede recibir las consultas).

Para conexiones HTTP, un balanceador de carga basta para manejar el enrutamiento de los servicios.



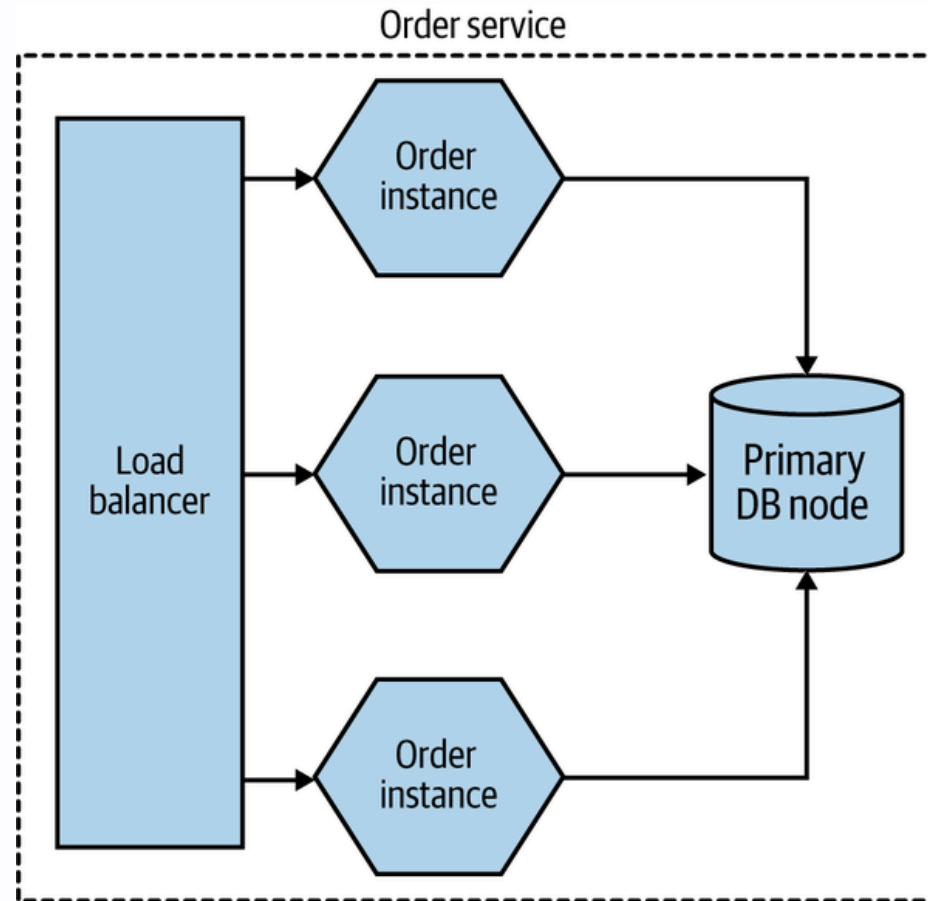
La cantidad de réplicas va a depender de la aplicación y el uso. Además, por razones de robustez, hay que preocuparse que no todas las instancias estén en el mismo hardware (servidor o incluso datacenter).



Réplica de la base de datos

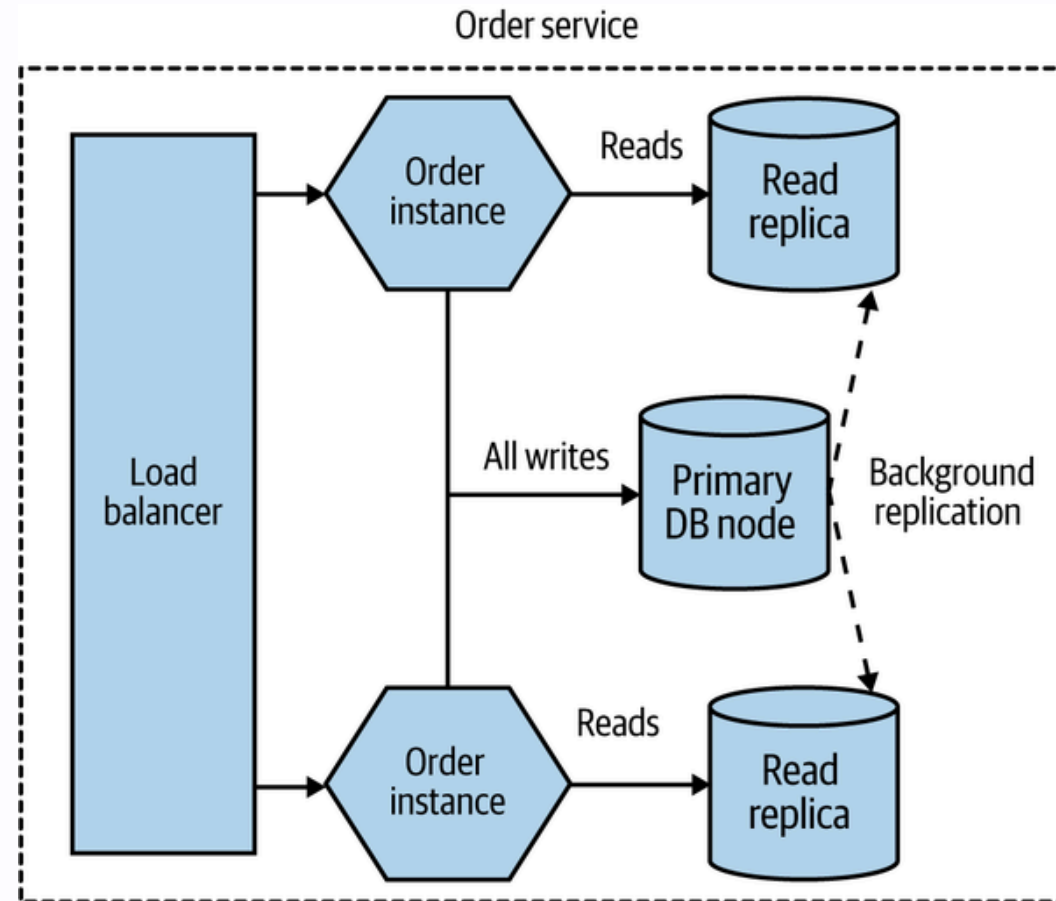
Como queremos que un microservicio oculte su gestión interna. Esto nos lleva a (siempre) repetir "no compartas base de datos".

Distintas réplicas de un servicio **pueden** compartir la misma instancia de una base de datos. Al ser el mismo servicio, la lógica de acceso y manipulación de datos, **siguen dentro del mismo servicio-lógico.**

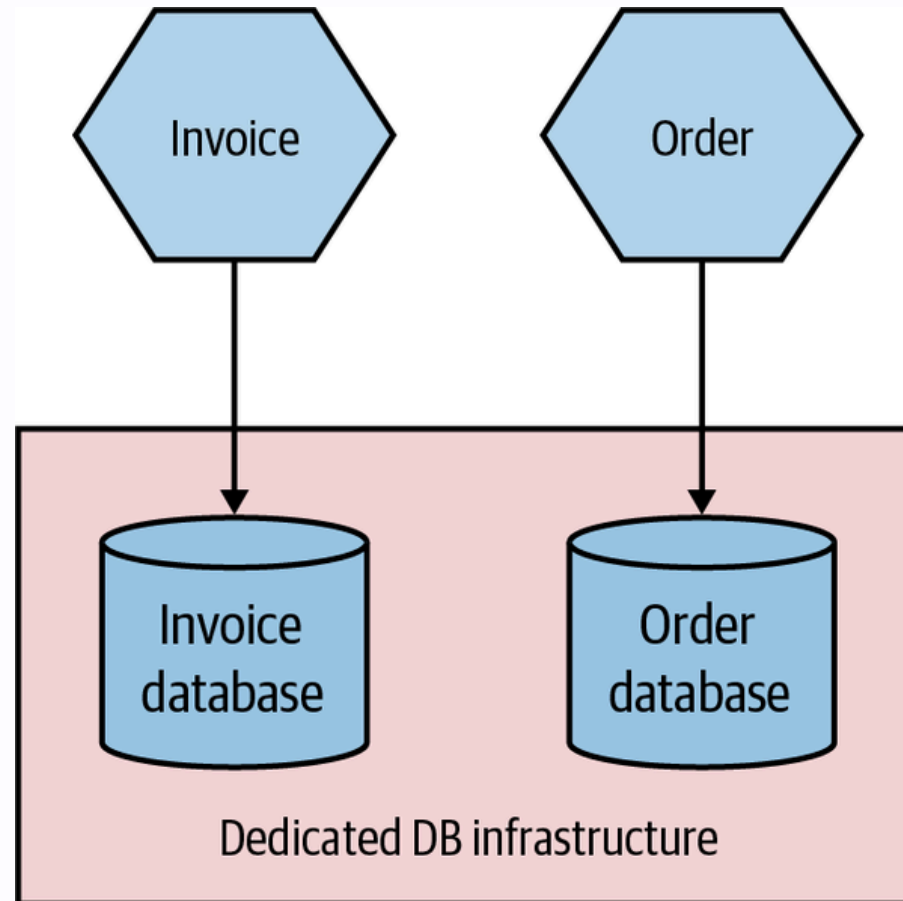


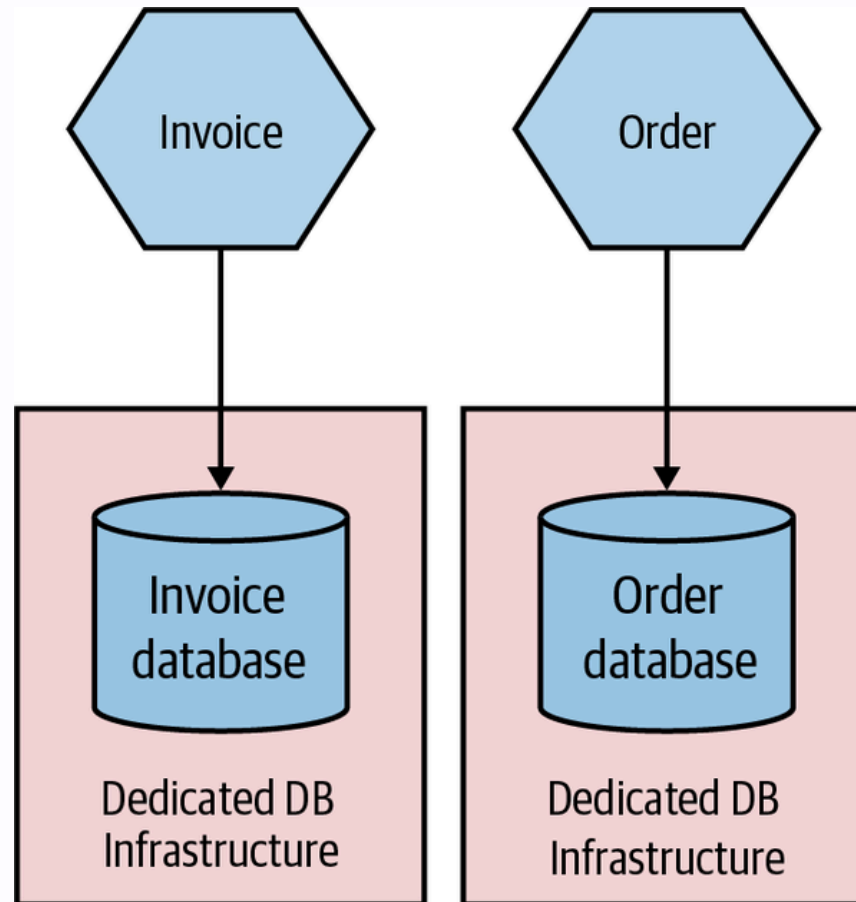
Escalar bases de datos

Existen diversos patrones de implementación de bases de datos replicadas, como tener réplicas *read-only*.



Otra decisión habitual es si aislar o no la base de datos en su propia infraestructura. Aunque estén en la misma infraestructura física, siguen siendo bases de datos lógicas distintas.

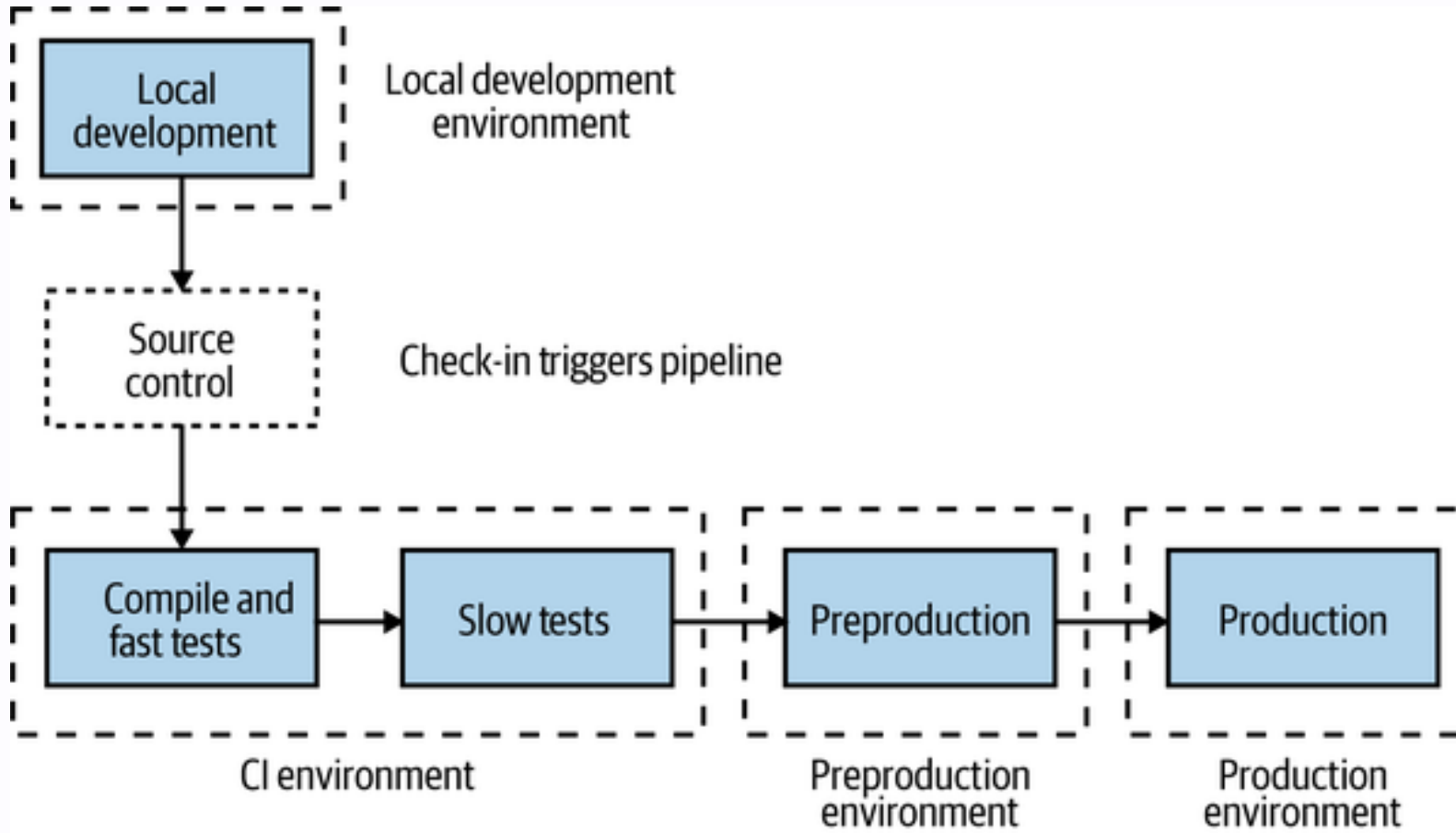




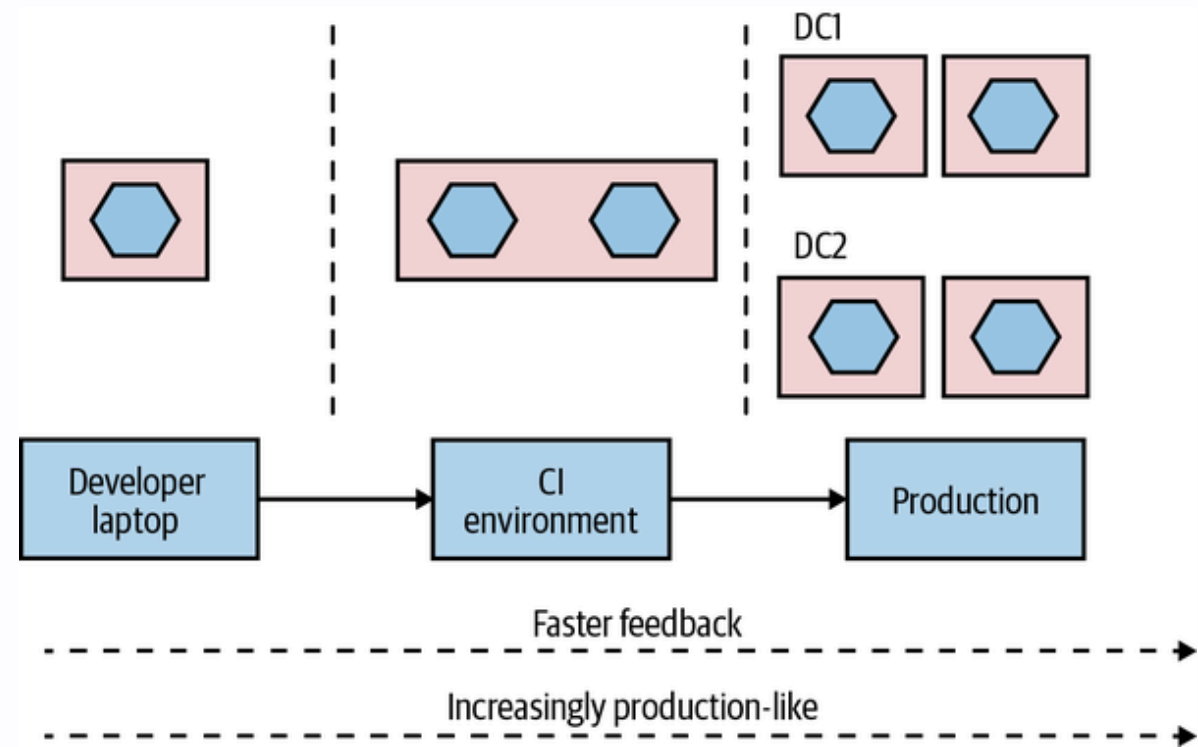
Entornos

Cuando despliegas un sistema, corre en un entorno. Distintos entornos pueden servir para distintos propósitos.

Uno puede desplegar la aplicación para realizar desarrollo local, con objetivo de realizar pruebas, como una beta para usuarios, como ambiente productivo, etc.



La integración continua (CI) es fundamental para subir versiones más rápido, probar antes y tener feedback temprano.



Entonces... Un microservicio puede estar en múltiples entornos y el número de réplicas de cada uno de estos microservicios puede variar en cada entorno dependiendo de las necesidades.



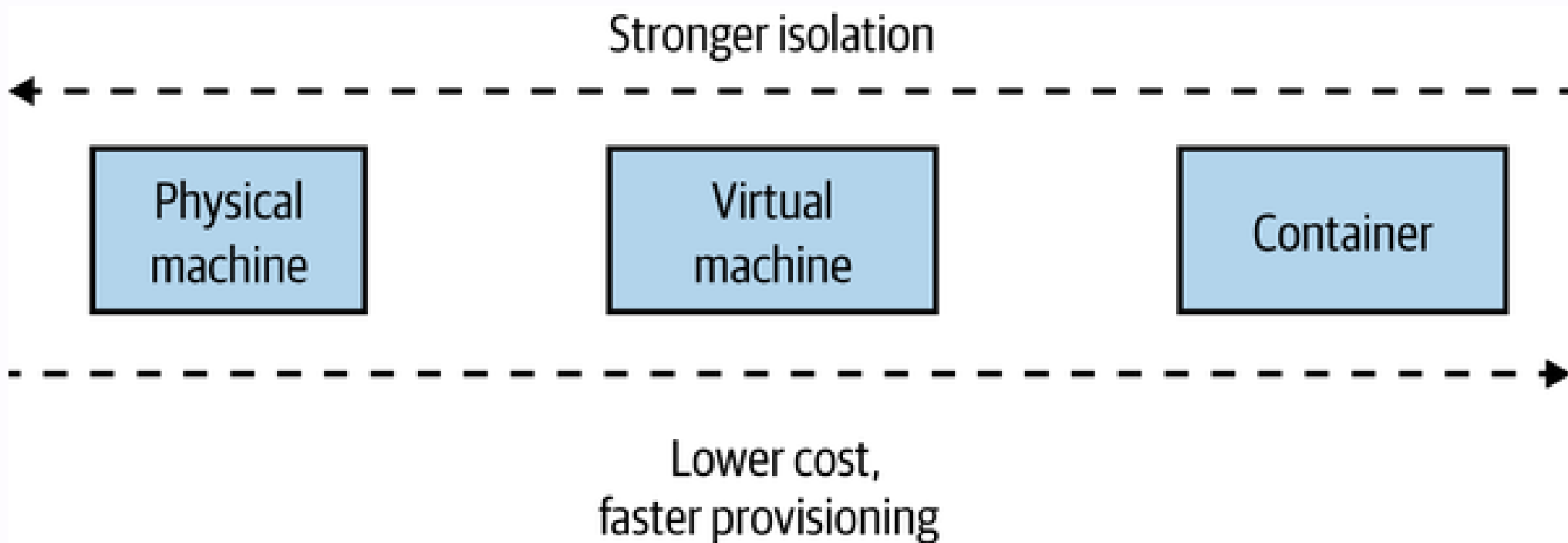
Principios del despliegue de microservicios



Ejecución aislada

Cada microservicio debe correr en una máquina (lógica) distinta. Cada servicio puede tener requerimientos distintos (y contradictorios), depender de equipos distintos y/o tener problemas que afecten a la máquina en su conjunto.

El aislamiento puede darse en máquinas (físicas) distintas, máquinas virtuales o contenedores.





Foco en la automatización

A medida que agregas más microservicios, tienes más partes que te tienes que ocupar. En vez de agregar más personas que administren cada microservicio, el foco debe estar puesto en la automatización.

Integración continua, escalamiento automático y migraciones de esquemas en base de datos, son algunos ejemplos de automatización.

Infraestructura como código

La infraestructura como código es llevar conceptos del desarrollo de software al despliegue de sistemas. Al definir la infraestructura mediante código, esta configuración pasa por control de versiones, se prueba y repetirse como parte de un proceso.

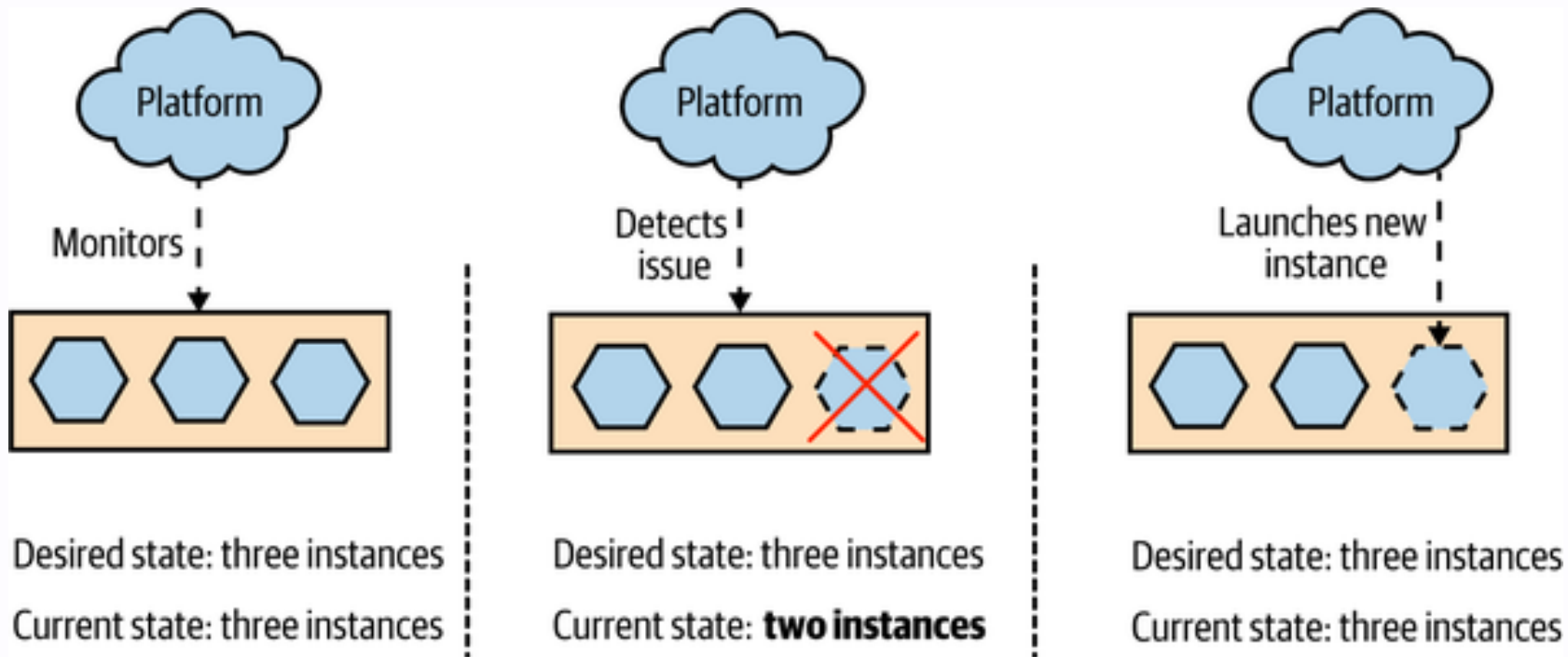
Cero tiempo de caída

Implementar sistemas que al ser actualizados no tengan tiempo de caída permite actualizaciones más constantes y, por ende, *feedback* temprano.

Gestión del estado deseado

Se especifica a la plataforma los requerimientos para la aplicación, y estos requerimientos se mantienen sin intervención humana. Cualquier problema, el sistema realiza los pasos para devolver el sistema al estado deseado.

Quita presión importante en la revisión y administración del sistema.





Opciones de despliegue

Máquinas físicas

En casos extraordinarios te vas a encontrar desplegando tus microservicios directamente en una máquina física. Sin otra capa de abstracción (contenedores o virtualización).

Esto produce sub-utilización de recursos y, habitualmente, mayor complejidad en el despliegue.

Máquinas virtuales

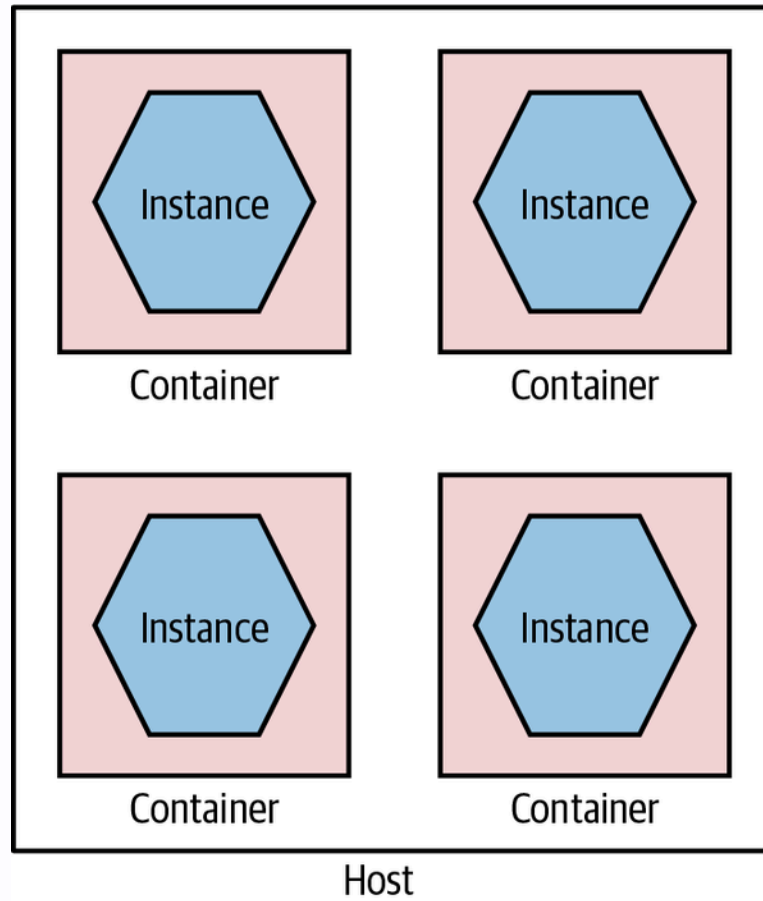
La virtualización nos permite dividir una máquina física en varias más pequeñas y virtuales.

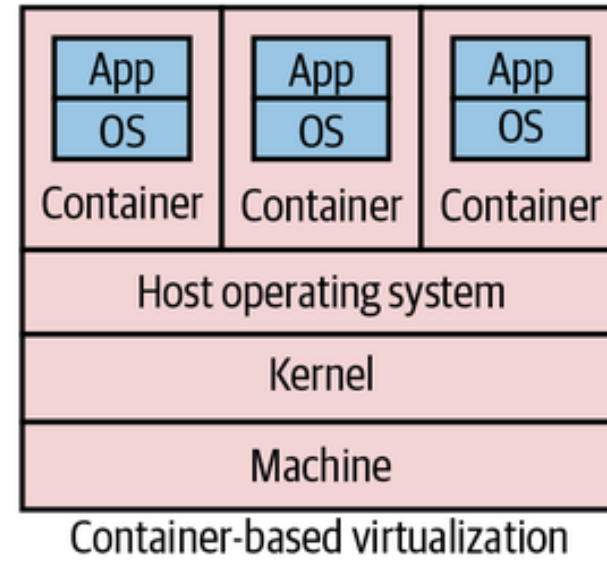
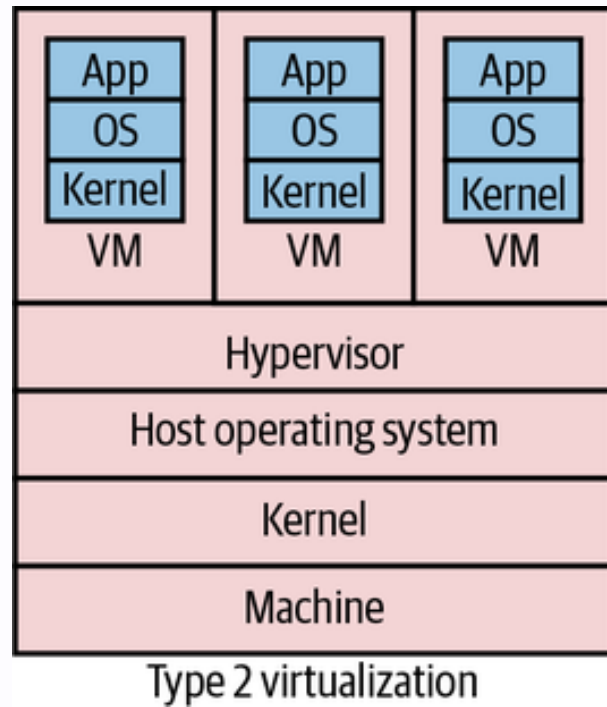
Tiene un costo importante el modelo de una máquina virtual por servicio. Además, mantienen parte de los problemas de las máquinas físicas en cuanto a la sub-utilización de recursos.



Contenedores

Los contenedores, popularizados por Docker con el apoyo de Kubernetes como orquestador, se ha convertido en la opción por defecto para ejecutar microservicio a escala.



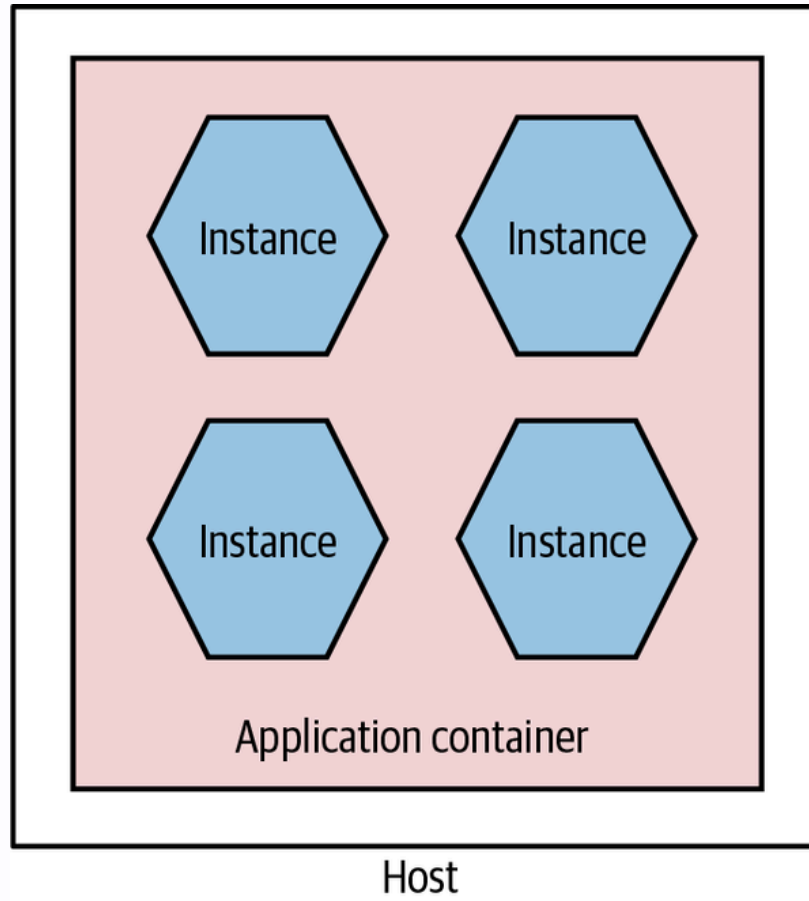




Contenedores de aplicaciones

Ciertos lenguajes o *frameworks* tienen sus propias formas de empaquetar el sistema; IIS en .NET o las *Java applications* son ejemplos de esto.

Puede traer beneficios en reducir el *overhead* de los contenedores.



Inevitablemente se limita la elección de tecnología. Habitualmente, se ejecutan con sistemas propios de automatización y despliegue.

Algunos contenedores de aplicaciones comparten recursos (memoria, cpu) entre instancias, reduciendo la capacidad de monitorearlos y su aislamiento.

La dependencia de un sistema como la JVM hace que cualquier cambio, implique reiniciar el sistema completo. No están realmente aisladas las instancias.

Platform as a Service (PaaS)

Plataformas de alto nivel de abstracción, algunas veces para tecnologías específicas, que se encargan del despliegue de la aplicación.

Más se encargue el PaaS, menor control tienes (y menos problemas puedes arreglar). Es parte del *trade-off*.

 Launch App < Back to Apps Home

Pricing   USAGE \$0.00 

Step 1 of 4

Select a repository

 GitHub

Repository

Choose a repository

Not seeing the repositories you expected here? [Configure your GitHub permissions](#)

Next

Try out one of our demo apps [View source on GitHub](#)

 React

 Next.js

 Ruby

 Ruby on Rails

 Jekyll

 Golang

 Node.js

 Hexo

 Gatsby

 PHP

 Laravel

 Hugo

 Python

 Vue.js

 Nuxt.js

 Static Assets

 Dockerfile



Function as a Service (FaaS)

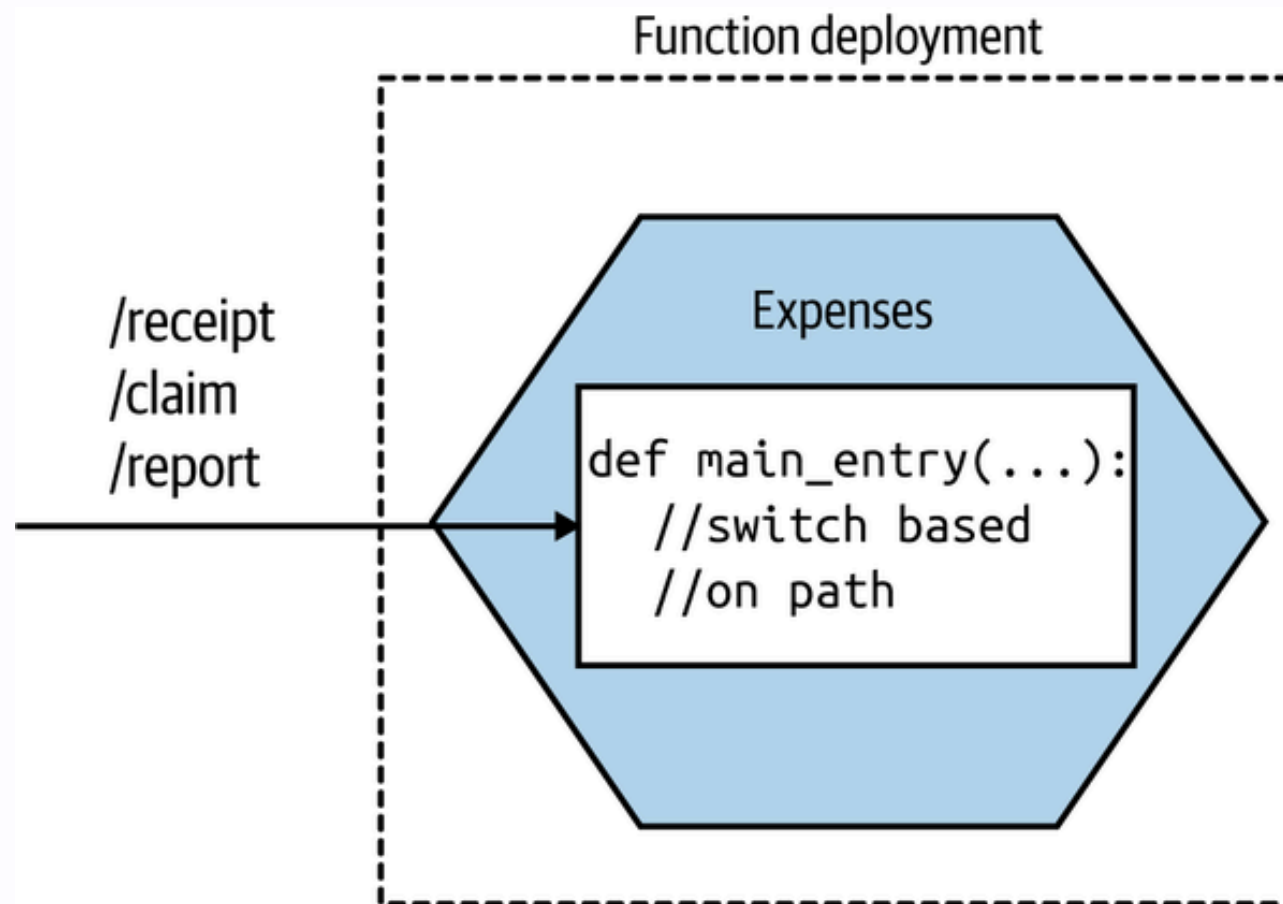
La frase "*serverless*" no significa que los servidores ya no estén involucrados. Significa que los desarrolladores ya no tienen que pensar sobre ellos.

Se despliega un servicio (la 'función'), el cual esta inactivo hasta que un evento pasa que gatilla la función. Ese evento puede ser un mensaje en una cola, una llamada HTTP o cualquier otra cosa. El código corre hasta que termina y se apaga.

Pueden ejecutarse múltiples funciones en paralelo y solo se te cobra por lo que se usa.

Por su naturaleza, FaaS tiende a entregar menor control sobre la ejecución, tiempo, recursos y limita que lenguajes/frameworks se pueden utilizar.

Además, algunos *runtimes*, les toma tiempo ejecutar una nueva versión, haciendo más lenta las llamadas.



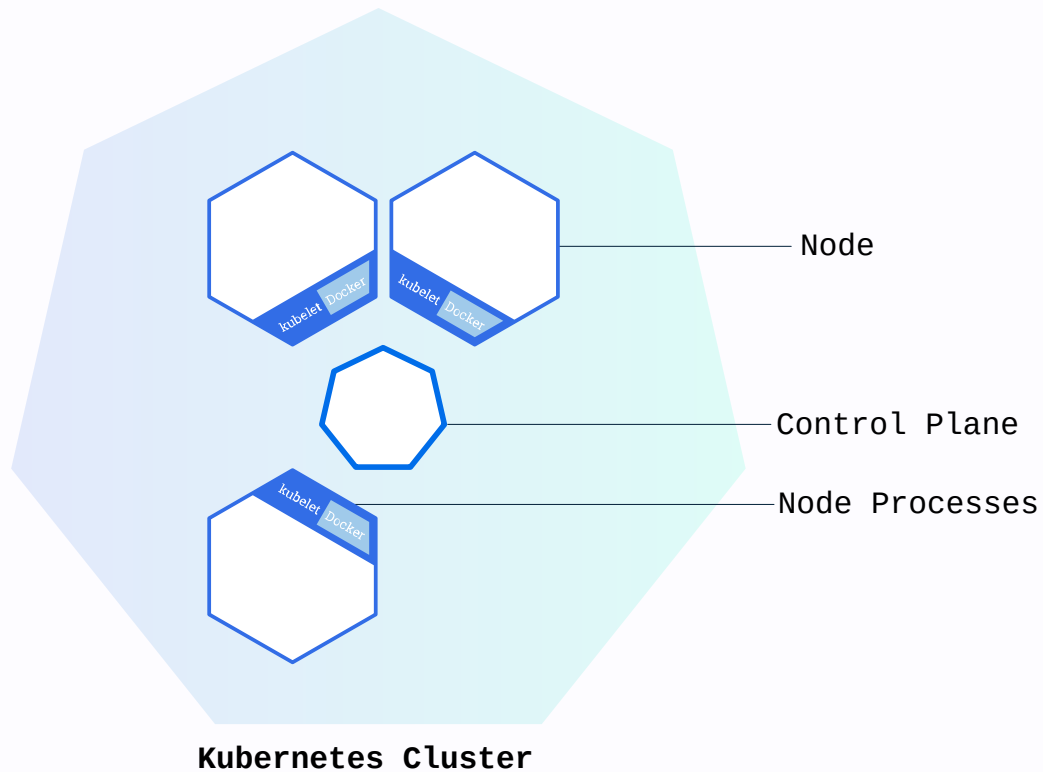
**Entonces... ¿Cómo despliego
mi aplicación? No existe
respuesta correcta, es un
constante trade-off entre
control/aislamiento vs
facilidad de despliegue/costos.**

Kubernetes (k8s)

Nuestros sistemas requieren estar desplegados en múltiples máquinas, para lo cual requerimos un sistema de orquestación de contenedores que se preocupe de correr (cuando lo requiramos) nuestros servicios, dejarlos disponibles en la red y manejar sus estados.

Kubernetes es la opción por defecto.

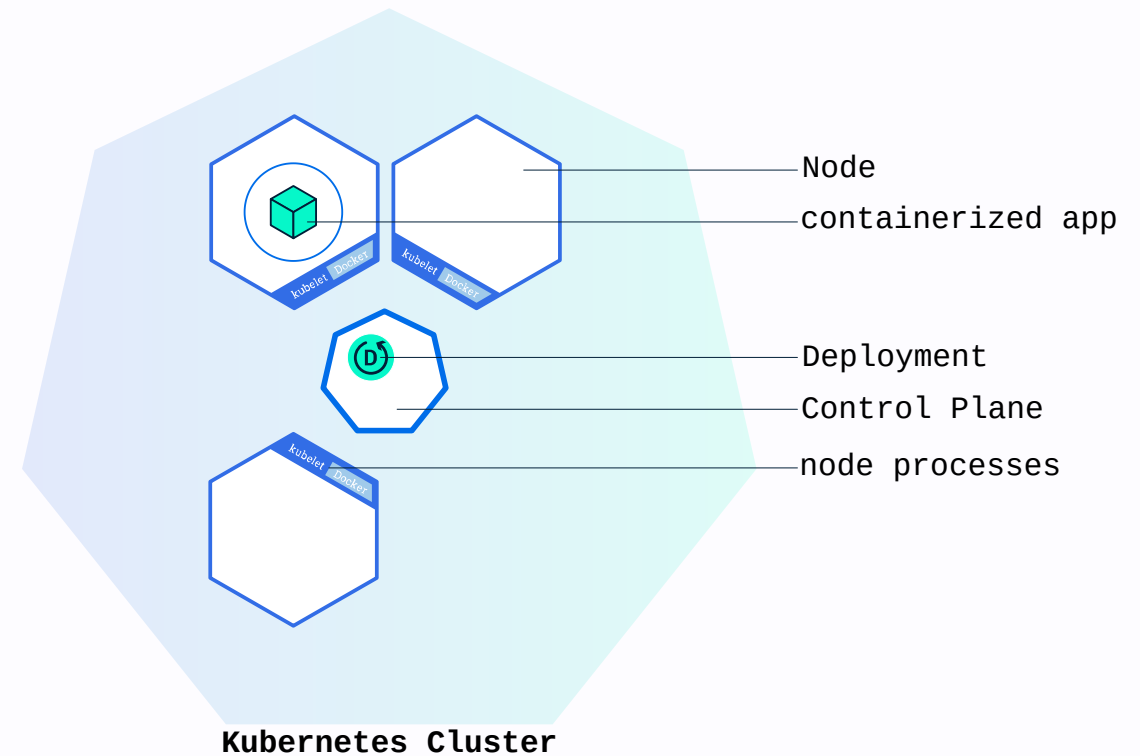
Kubernetes coordina un cluster de alta disponibilidad a través de computadores que están conectados y trabajan como una única unidad. Esto lo hace automatizando la distribución y organización de los contenedores en la forma más eficiente.

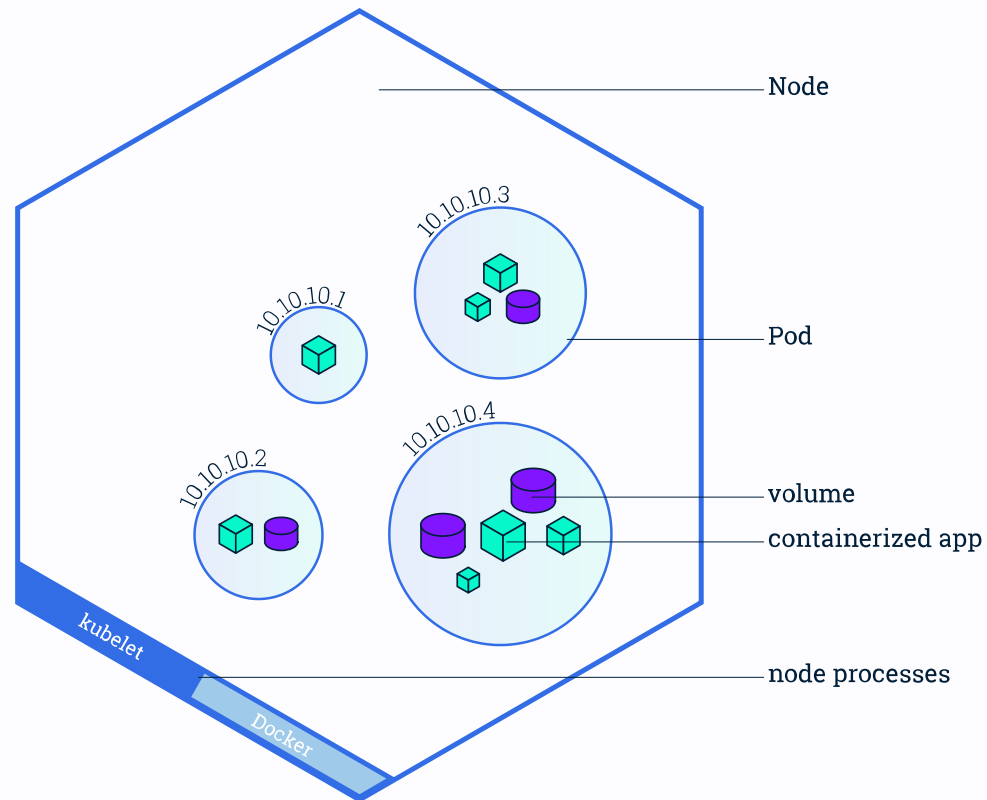


Un **cluster** tiene dos tipos de recursos:

- **Sistema de control:** administrar el clúster.
- **Nodos:** máquinas virtuales o físicas donde se ejecutan los contenedores.

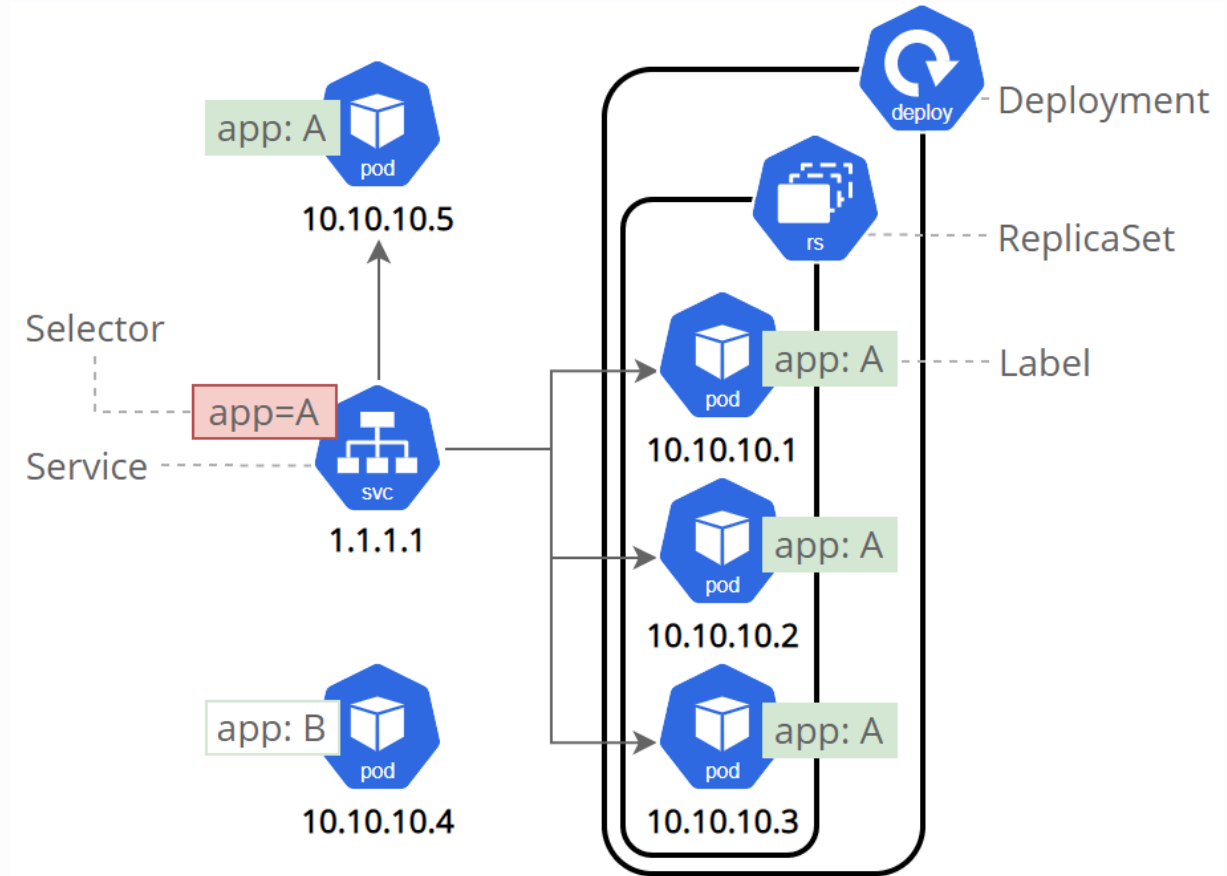
Los **Deployment** son instrucciones para k8s de como crear y actualizar las instancias de la aplicación. Se preocupa que, incluso si se borra o falla un nodo, el sistema se recupere solo.





Un **Pod** es un grupo de contenedores. Incluye sus recursos como almacenamiento (**volumes**), **IP única** en el cluster e información de como correr cada contenedor.

Un **Service** es una abstracción que permite definir como acceder a un conjunto de **Pods**.



Los **Services** pueden ser:

- **ClusterIP**: expone un servicio internamente.
- **NodePort**: expone el servicio externamente
`<NodeIP>:<NodePort>` .
- **LoadBalancer**: un balanceador de carga, al cual se asigna una IP externa.
- **ExternalName**: redirige el contenido del servicio al campo `externalName` (ej. `foo.bar.example.com`).

Ingress

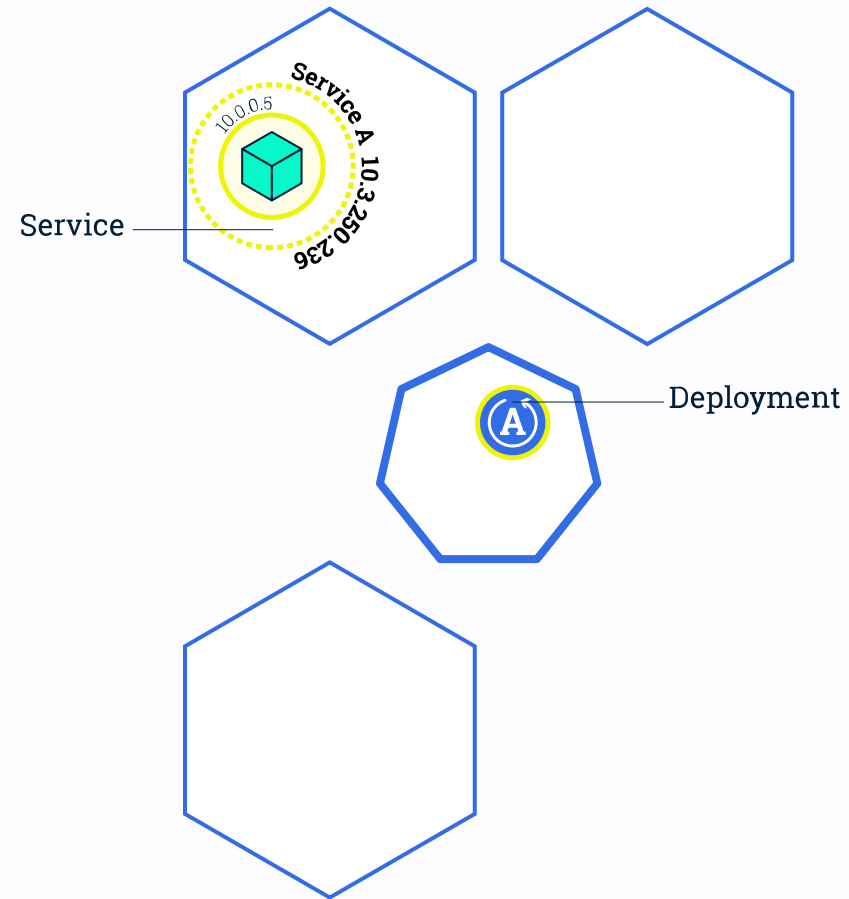
Un **Ingress** expone servicios HTTP/HTTPS hacia afuera del cluster, con reglas por host y por ruta. No basta con definirlo: necesita un *ingress controller* corriendo en el cluster (nginx es el más usado).

La clase de controlador se declara con `spec.ingressClassName`.

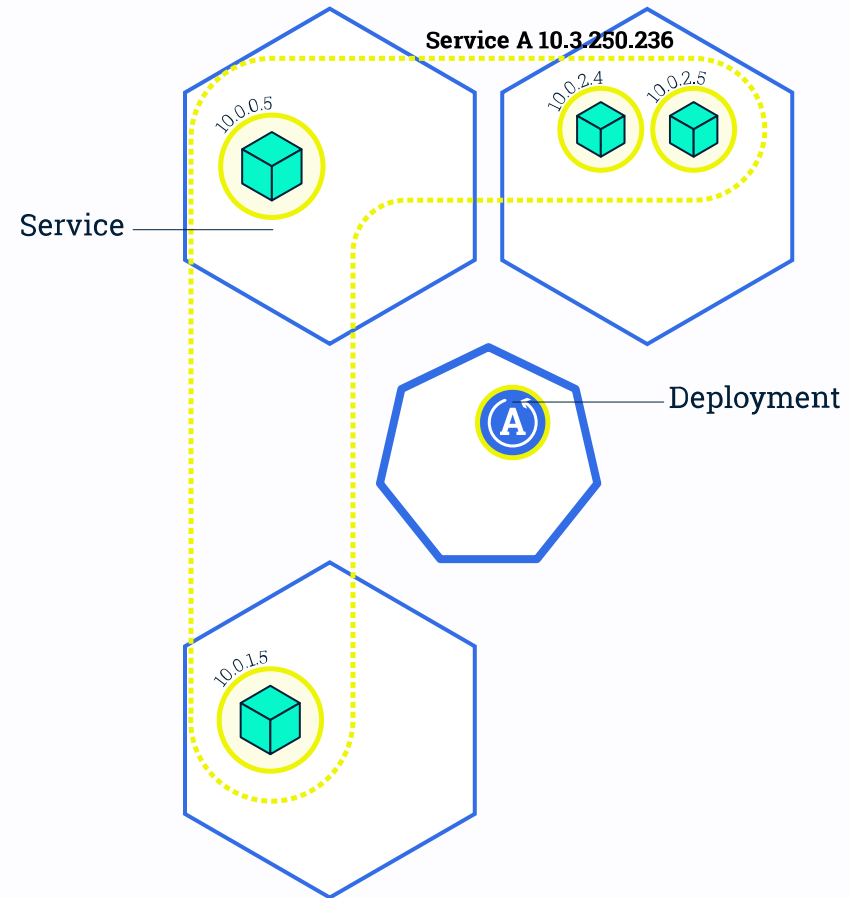
TLS con cert-manager

cert-manager obtiene y renueva certificados de forma automática, por ejemplo contra Let's Encrypt. Se define un **Issuer** con los datos de la autoridad certificadora y el Ingress declara en **spec.tls** los hosts y el secreto donde queda el certificado. Con eso, HTTPS deja de ser un trámite manual.

Escalar un servicio (1)



Escalar un servicio (2)





HorizontalPodAutoscaler (HPA)

Escalar a mano no es sostenible. El **HPA** ajusta solo la cantidad de réplicas de un Deployment según métricas, típicamente uso de CPU.

Requiere que los contenedores declaren `resources.requests` : sin eso, k8s no tiene contra qué calcular el porcentaje de uso.

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: api-gateway-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: api-gateway-deployment
  minReplicas: 2
  maxReplicas: 5
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
```

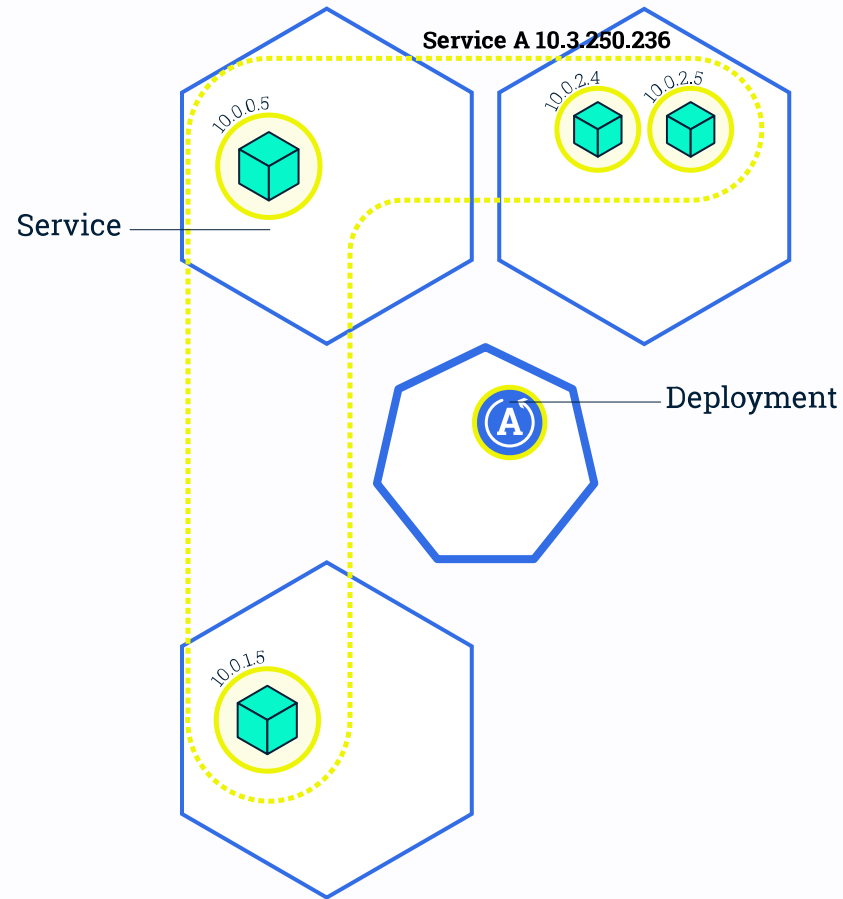


Escalar consumidores de colas

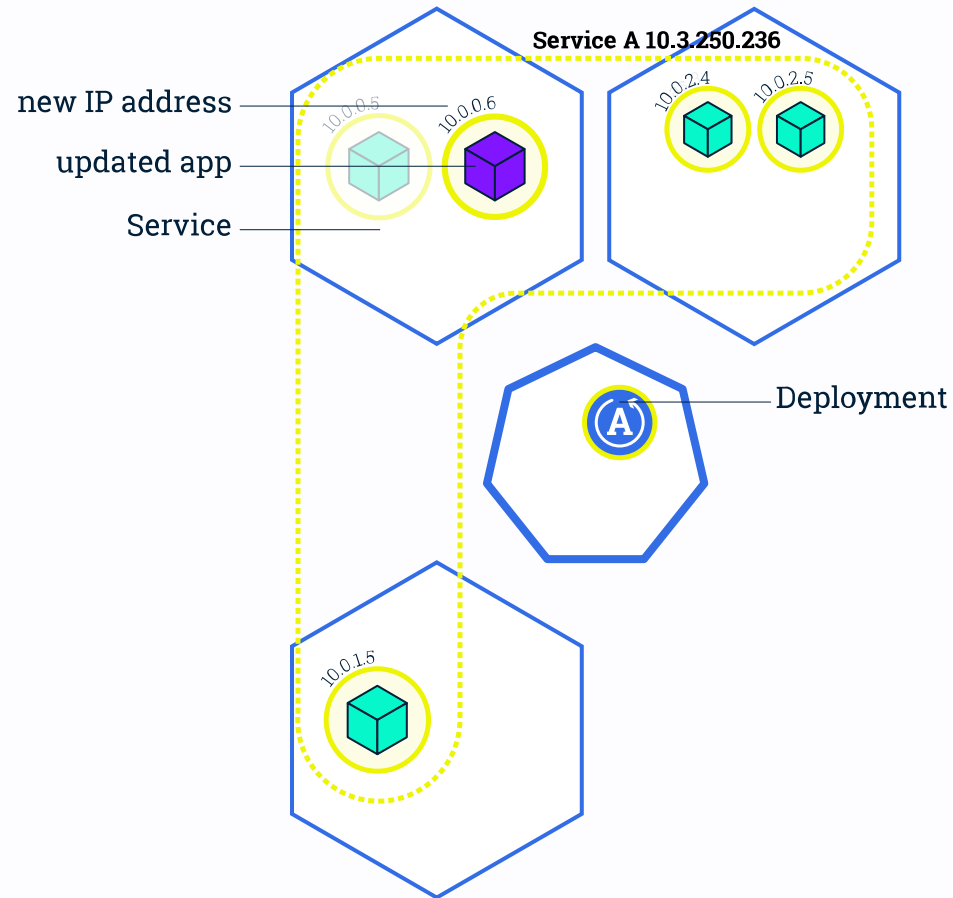
Para *workers* que procesan colas, la CPU es mala señal: la señal correcta es el largo de la cola.

Si las tareas entran más rápido de lo que se procesan, la cola crece sin control (*backpressure*). La respuesta es escalar los consumidores según los mensajes pendientes. Herramientas como **KEDA** permiten a k8s escalar con métricas externas (RabbitMQ, Kafka, etc.).

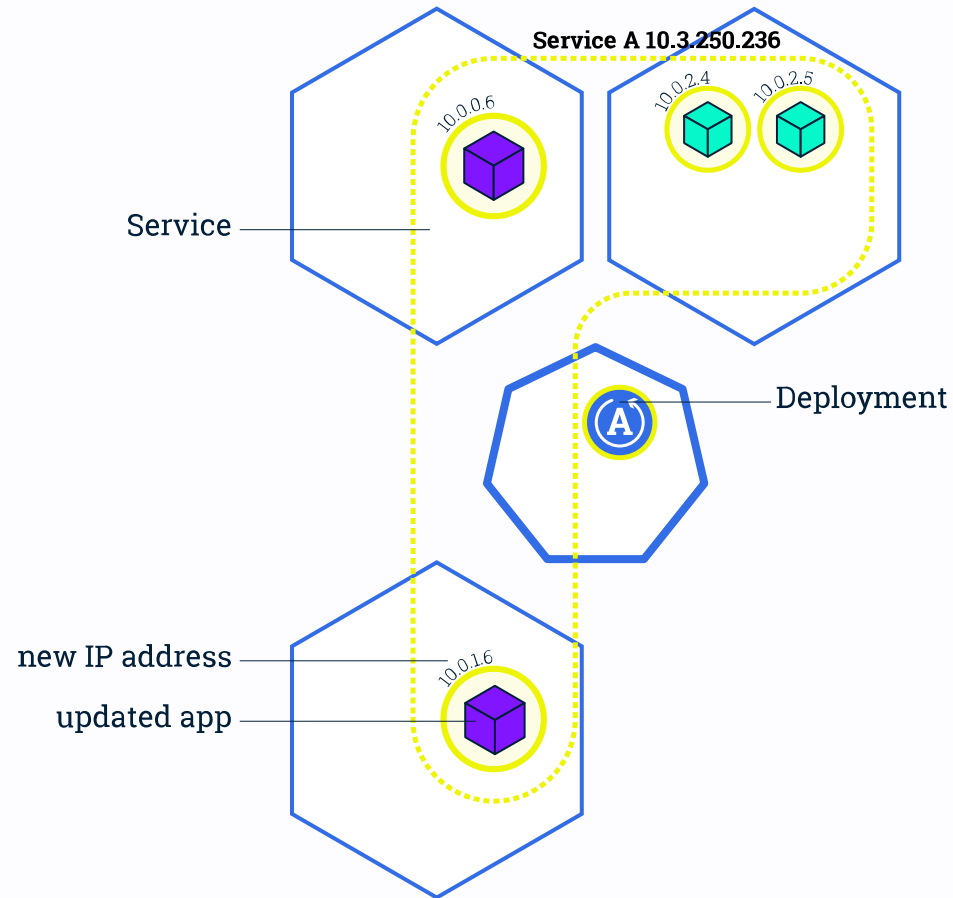
Actualizar un servicio (1)



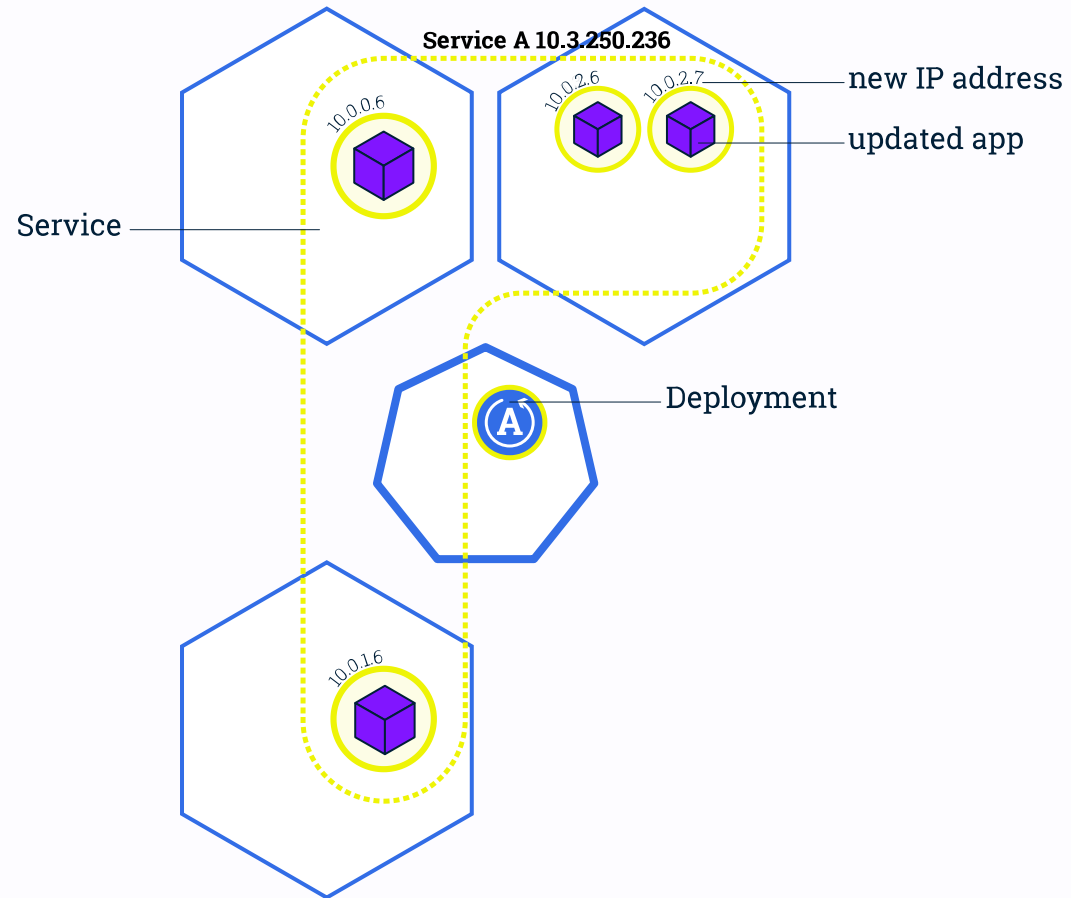
Actualizar un servicio (2)



Actualizar un servicio (3)



Actualizar un servicio (4)



Ejemplo: `./demo_05`

- Un *middleware* con archivos de configuración para levantarlo en *docker-compose* y *Kubernetes*.
- El archivo de configuración de la Integración Continua (`.gitlab-ci.yml`).



Tarea

Sube tu microservicio al cluster Kubernetes, idealmente mediante Integración Continua. Debe implementar una URL pública y *autoscaling*.



Material complementario

- Building microservices: Designing fine-grained systems, Sam Newman (2021). O'Reilly. Capítulo 8.
- [Learn Kubernetes Basics](#)